

O S

操作系统

高茜



O S

Chapter 2

操作系统用户界面



O S

1

简介及一般用户的输入输出界面

2

命令控制界面、图形控制界面

3

Linux与Windows的命令控制界面

4

系统调用

5

Linux与Windows的系统调用



计算机系统的用户

普通
用户



管理
员用
户

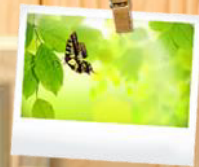
被服务者（使用
和管理计算机应
用程序的用户）



程序开发人员（使用
操作系统所提供的编
辑功能开发新的应用
程序，完成用户所要
求的服务）



计算
机系
统的
用户



操作系统用户界面

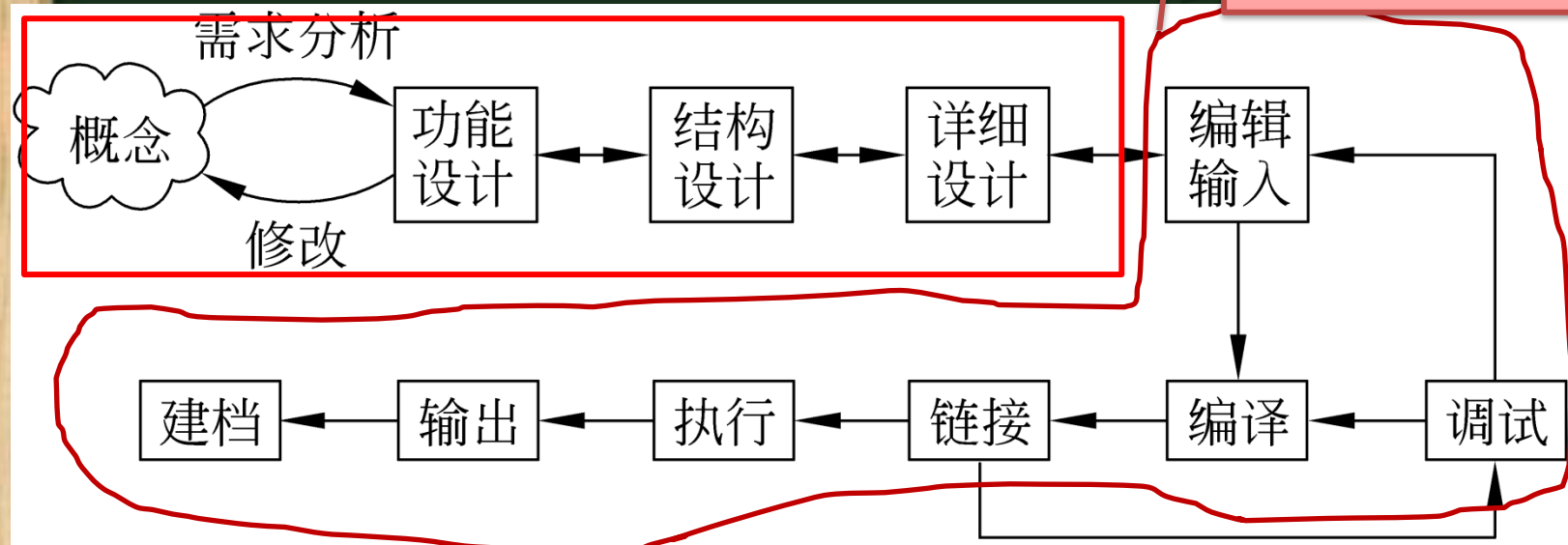
- 操作系统为普通用户和管理员提供的界面是由一组不同形式表示的操作命令组成。
- 不同计算机操作系统为用户提供的用户操作命令和表现形式不同。
- 不同时期的操作系统为用户提供的操作命令和表现形式也不同。
- 操作系统为编程人员提供的界面是系统调用。



作业的

直到编辑为止的各步都可认为是可由人工独立完成的（尽管也有许多支撑软件存在）

从编辑输入开始的以下各步却是在用户的要求控制下由计算机完成



作业的定义

- 在一次应用业务处理过程中，从输入开始到输出结束，用户要求计算机所做的有关该次业务处理的全部工作称为一个**作业**。
- 作业由不同的**顺序相连的作业步**组成。作业步是在一个作业的处理过程中，计算机所做的相对独立的工作。

作业的定义

- 从系统的角度看，作业是一个比程序更广的概念。它由程序、数据和作业说明书组成。系统通过作业说明书控制文件形式的程序和数据，使之执行和操作。
- 在批处理系统中，作业是抢占内存的基本单位。也就是说，批处理系统以作业为单位把程序和数据调入内存以便执行

作业组织

➤一个作业
含一个程序
完成用户所
➤作业说明
中生成一个
格。该表格
执行优先级
并分配资源
执行等。

作业基本情况描述

- 用户名
- 作业名
- 使用语言名
- 允许最大处理时间
- 等等

作业控制描述

- 控制方式
- 操作顺序
- 出错处理
- 等等

作业资源要求描述

- 要求处理时间
- 内存空间
- 外设类型和数量
- 处理机优先级
- 库函数或实用程序
- 等等

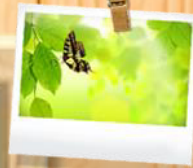
至少包
和数据
在系统
B)的表
时间和
要求，
装入和

一般用户的输入输出方式



联机输入输出方式

➤大多用在交互式系统中, 用户和系统通过交互会话来输入作业。外围设备直接和主机相连接。一台主机可以连接一台或多台外围设备。

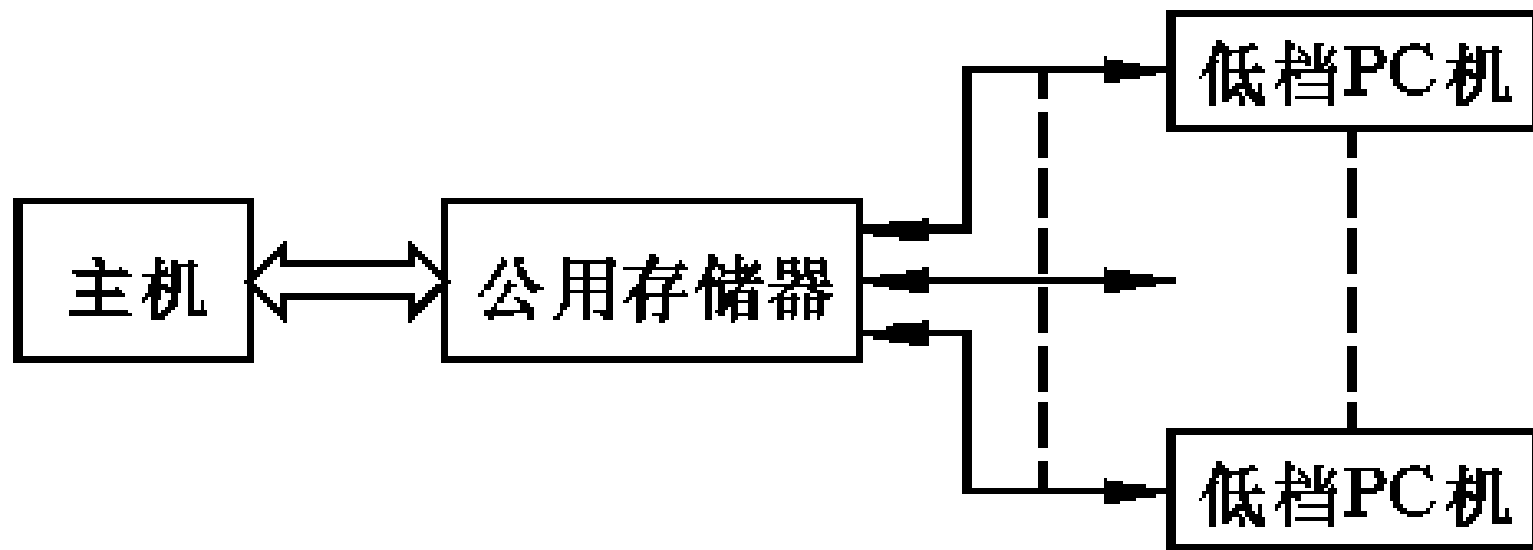


脱机输入输出方式

➤ 又称为**预输入方式**。脱机输入方式利用低档个人计算机作为外围处理机进行输入处理。在**低档个人机上**，**用户通过联机方式把作业首先输入到后援存储器**，例如磁盘或磁带上；然后，用户把装有输入数据的后援存储器拿到主机的高速外围设备上和主机连接，从而在较短的时间内完成作业的输入工作。

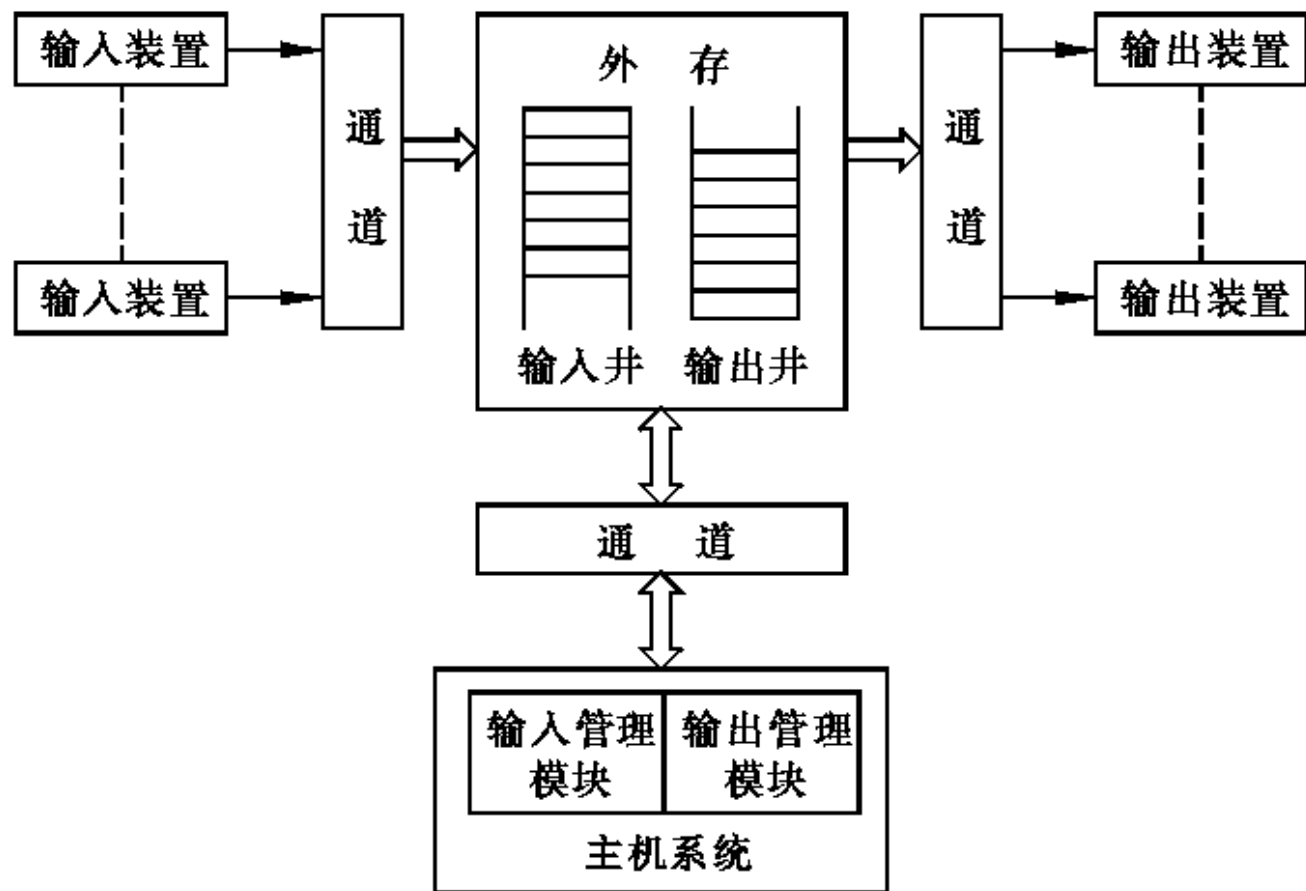


直接耦合方式



SP

系统



网络联机方式

➤ 网络联机方式以上述几种输入方式为基础。当用户需要把在计算机网络中某一台主机上输入的信息传送到同一网中另一台主机上进行操作或执行时，就构成了网络输入方式。因为网络输入方式涉及到不同计算机间的通信问题，且该问题的讨论已超出了本书的范围，因此，本书中不准备进一步讨论。



O S

1

简介及一般用户的输入输出界面

2

命令控制界面、图形控制界面

3

Linux与Windows的命令控制界面

4

系统调用

5

Linux与Windows的系统调用



OS 启动后

各种应用程序

局部描述符表 IDT

全局描述符表 GDT

管理内存的 mem_map
位图数据结构

OS 代码

应用程序
是如何穿
入到 OS 代
码部分来
使用操作
系统呢？

- 1、存放目前还有多少容量的内容可供使用，哪些内存是空闲的可使用的，哪些是已经被使用过的
- 2、数组中的每个项用来表示内存中一段固定大小的内存
- 3、初始化时将所有数组项中的内容都设置为 0

什么是操作系统接口?

- 仍然从常识开始——日常生活中有很多接口：电源插座；汽车油门…

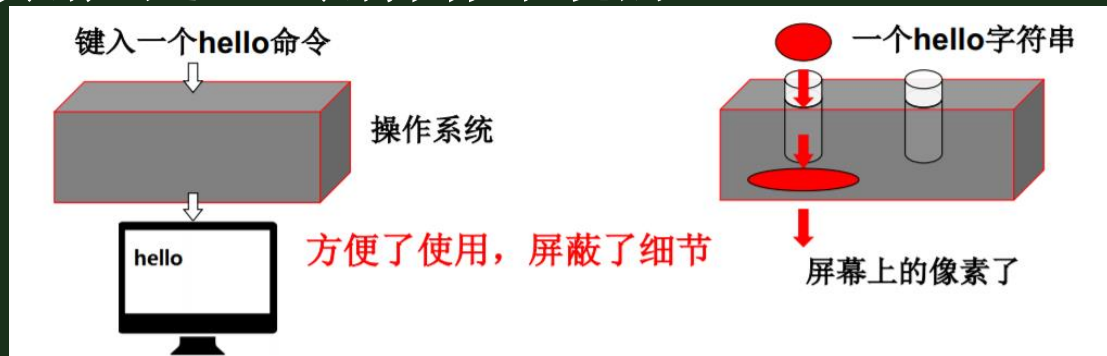


- 什么是接口？
连接两个东西、信号转换、屏蔽细节…

什么是操作系统接口?

- 连接上层用户和操作系统软件
- 问题：操作系统直接面对用户吗?

即用户是怎么用操作系统的?...



- 由上图可知，用户可以使用“命令”来使用操作系

什么是操作系统接口?

➤ 用户如何使用计算机?

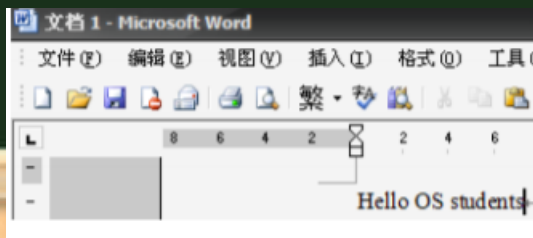
◆ 命令行:

```
cst:/home/lizhijun# ./output "hello"  
ECHO:hello
```

◆ 图形按钮:



◆ 应用程序:



命令控制界面

- OS启动之后，会打开一个桌面或shell (Linux)

```
int main(int argc, char * argv[ ])  
{ char cmd[20];  
  while(1) { scanf( "%s" , cmd);  
              if(!fork()) {exec(cmd);}  
              else {wait();} } //while(1)  
}
```

命令控制界面

- 命令是什么？命令输入后发生了什么？

```
cst:/home/lizhijun# ./output "hello"  
ECHO:hello
```

```
#include <stdio.h>
```

```
int main(int argc, char * argv[])
```

```
{ printf("ECHO:%s\n", argv[1]); }
```

gcc -o output output.c

./output "hello"

- 命令实际上就是一段用C语言写的程序，只是在这些程序中加了一些非常重要的函数，编译之后形成可执行文件，进而被执行。

使用操作命令进行作业控制有两种方式

脱机方式

用户将作业的执行顺序和出错处理方法一并以作业控制说明书的方式或命令文件方式提交给系统，由系统按照作业说明书或命令文件中所规定的顺序控制作业执行。执行过程中，用户无法干涉，只能等待作业正常执行结束或出错停止之后查看执行结果或出错信息，以便修改作业内容或控制过程。

联机方式

不要求用户填写作业说明书，系统只为用户提供一组键盘或其他操作方式的命令。用户使用系统提供的操作命令和系统会话，交互地控制程序执行和管理计算机系统。

其工作过程是，用户在系统给出的提示符下敲入特定的命令，系统在执行完该命令后向用户报告执行结果；然后，用户决定下一步的操作。如此反复，直到作业执行结束。

聯

C:\WINDOWS\system32\cmd.exe

```
C:\>cac ls d:tools
```

```
D:\tools BUILTIN\Administrators:F
```

```
BUILTIN\Administrators:(OI)(CI)(IO)F
```

C:\WINDOWS\system32\cmd.exe - cac ls d:tools /p administrator:c

```
C:\>cac ls d:tools /p administrator:c
```

```
是否确定(Y/N)?
```

替换指定用户的访问权限

F表示完全控制，C表示更改，W表示写入

TEMP=C:\Users\茜\AppData\Local\Temp

联机控制方式的命令种类

某单位一台电脑要添加另一台ip为192.168.1.3电脑上共享名为hp1000的打印机.该电脑设置了密码限制访问未完全开放共享,因此添加该机器上的打印机时,需要先用NET USE命令开启共享资源。所需命令如下:

net use \\192.168.1.3\ipc\$ /user:username password

链接装配和执行编辑命令



主机和进程之间
之间的呼叫、连接以及断开等

资源要求
向系统申请资源

命令控制界面

➤ 联机控制方式大大地方便了用户。但在某些情况下，用户反复输入众多的命令也会浪费不必要的时间。因此，在现代操作系统中，大都提供批处理方式和联机控制方式。

命令控制界面

➤近年来，命令控制界面的人机交互方式发生了革命性变化。无论是Windows系列，还是UNIX系列的操作系统，它们的命令控制界面都是由多窗口的按钮式图形界面组成。在这些系统中，命令已被开发成一条条能用鼠标点击而执行的简单的**菜单**或小巧的**图标**。而且，用户也可以在提示符的提示下用普通字符方式输入各种命令。可以预计，计算机系统的命令控制界面将会越来越方便和越来越拟人化。

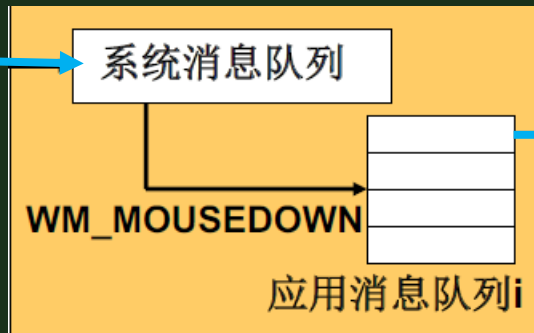


图形控制界面

- 图形按钮本质上是基于一个消息队列
- 点击鼠标或者按下键盘

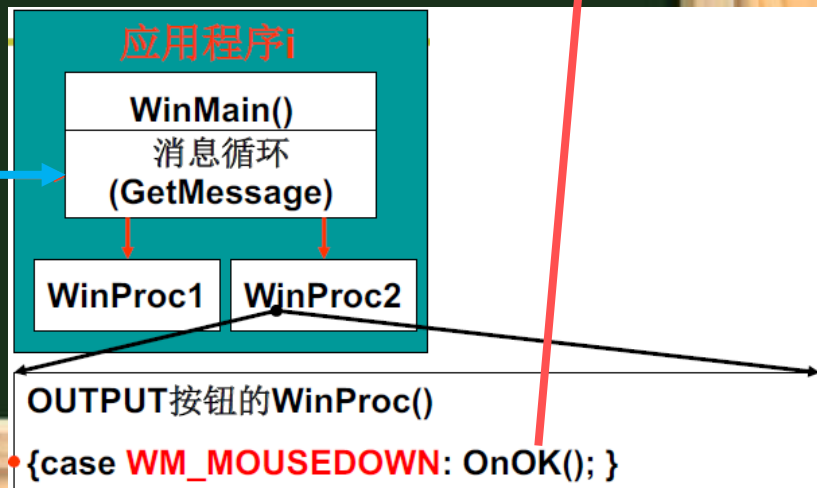
个消息队列，而应用程序则需要写一个系统调用，从消息队列中取出相应的消息。

硬件输入



图形界面：包括画图的C程序

```
void COutputDlg::OnOK()
{ GetDlgItemText(IDC_EDIT1,m_outStr);
  FILE *fp=fopen("d:\\out.txt","w");
  printf(fp,m_outStr,m_outStr.GetLength());
  fclose(fp);
}
```



操作系统接口

- 命令行：命令程序。
- 图形界面：消息框架程序+消息处理程序。
- 用户使用计算机：通过程序
- 操作系统接口连接的就是操作系统和应用程序
- 如何连接？——C程序。

普通C代码加上一些重要的函数

```
#include <stdio.h>

int main(int argc, char * argv[])
{ printf("ECHO:%s\n", argv[1]); }
```

操作系统接口

- 操作系统提供这样的重要函数。
- 本质上操作系统接口变现为函数调用，由于这些函数是由系统提供的，因此称为系统调用(system _call)。

O S

1

简介及一般用户的输入输出界面

2

命令控制界面、图形控制界面

3

系统调用

4

Linux与Windows的命令控制界面

5

Linux与Windows的系统调用



系统调用

- 系统调用是操作系统提供给编程人员的唯一接口。编程人员利用系统调用, 在源程序一级动态请求和释放系统资源, 调用系统中已有的系统功能来完成那些与机器硬件部分相关的工作以及控制程序的执行速度等。
- 系统调用像一个黑箱子那样, 对用户屏蔽了操作系统的具体动作而只提供有关的功能。事实上, 命令控制界面也是在系统调用的基础上开发而成的。



系统调用

➤ **POSIX**: Portable Operating System

Interface of Unix(IEEE 制定的一个标准族)。

分类	POSIX定义	描述
任务管理	fork	创建一个进程
	execl	运行一个可执行程序
	pthread_create	创建一个线程
文件系统	open	打开一个文件或目录
	EACCES	返回值, 表示没有权限
	mode_t st_mode	文件头结构: 文件属性

系统调用

设备管理

- 该类系统调用被用来请求和释放有关设备、以及启动设备等

文件管理

- 对文件的读、写、创建和删除等

进程创建

- 进程是一个在功能上独立的程序的一次执行过程。进程控制的有关系统调用包括进程创建、进程执行、进程撤消、执行等待和执行优先级控制等。

进程通信

- 该类系统调用被用在进程之间传递消息或信号。

存储管理

- 包括调查作业占据内存区的大小、获取作业占据内存区的始址等。

线程管理

- 包括线程的创建、调度、执行、撤销等。

系统调用

➤ 为了提供系统调用功能，操作系统内必须有**事先编制好的实现这些功能的子程序或过程**。显然，这些程序或过程是操作系统程序模块的一部分，且**不能直接被用户程序调用**。而且，为了保证操作系统程序不被用户程序破坏，一般操作系统都不允许用户程序访问操作系统的系统程序和数据。



系统调用

■ 实现一个whoami系统调用

- 用户程序调用whoami，一个字符串“lizhijun”放在操作系统中(系统引导时载入)，取出来打印
- 不能随意的调用数据，不能随意的jmp。
- 可以看到root密码，可以修改它...
- 可以通过显存看到别人word里的内容...

```
main()  
{ whoami();}
```

用户程序

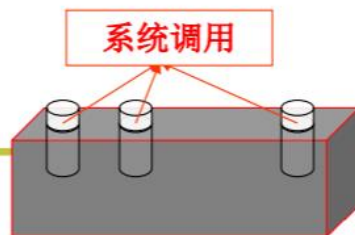
```
whoami()  
{  
    printf(100, 8);  
}
```

都在内存中，这内存不都是我买的吗...

100: "lizhijun"

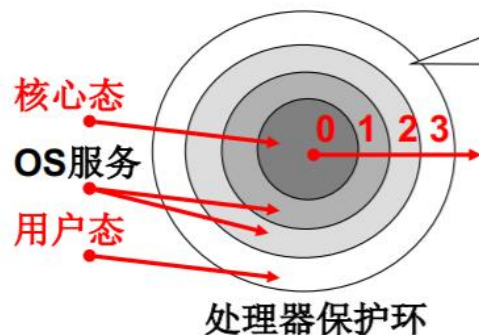


内核(用户)态, 内核(用户)段



■ 将内核程序和用户程序隔离!!!

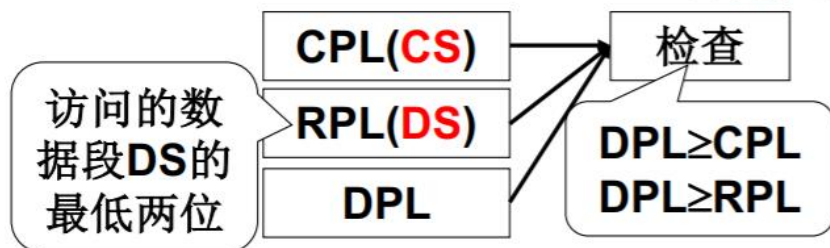
- 区分内核态和用户态: 一种处理器“硬件设计”



当前程序执行在什么态(哪层环)? 由于CS:IP是当前指令, 所以用CS的最低两位来表示: 0是内核态, 3是用户态

- 内核态可以访问任何数据, 用户态不能访问内核数据

- 对于指令跳转也一样实现了隔离...



如何才能进入系统内核呢？

- 硬件提供了“主动进入内核的方法”
- 对于Intel x86，那就是中断指令int

■ 系统调用的核心：

由谁做？库函数！

- (1) 用户程序中包含一段包含int指令的代码
- (2) 操作系统写中断处理，获取想调程序的编号
- (3) 操作系统根据编号执行相应代码

如何才能进入系统内核呢？

应用程序

调用printf(...)

C函数库

库函数printf(...)

库函数write(...)

OS内核

系统调用write(...)

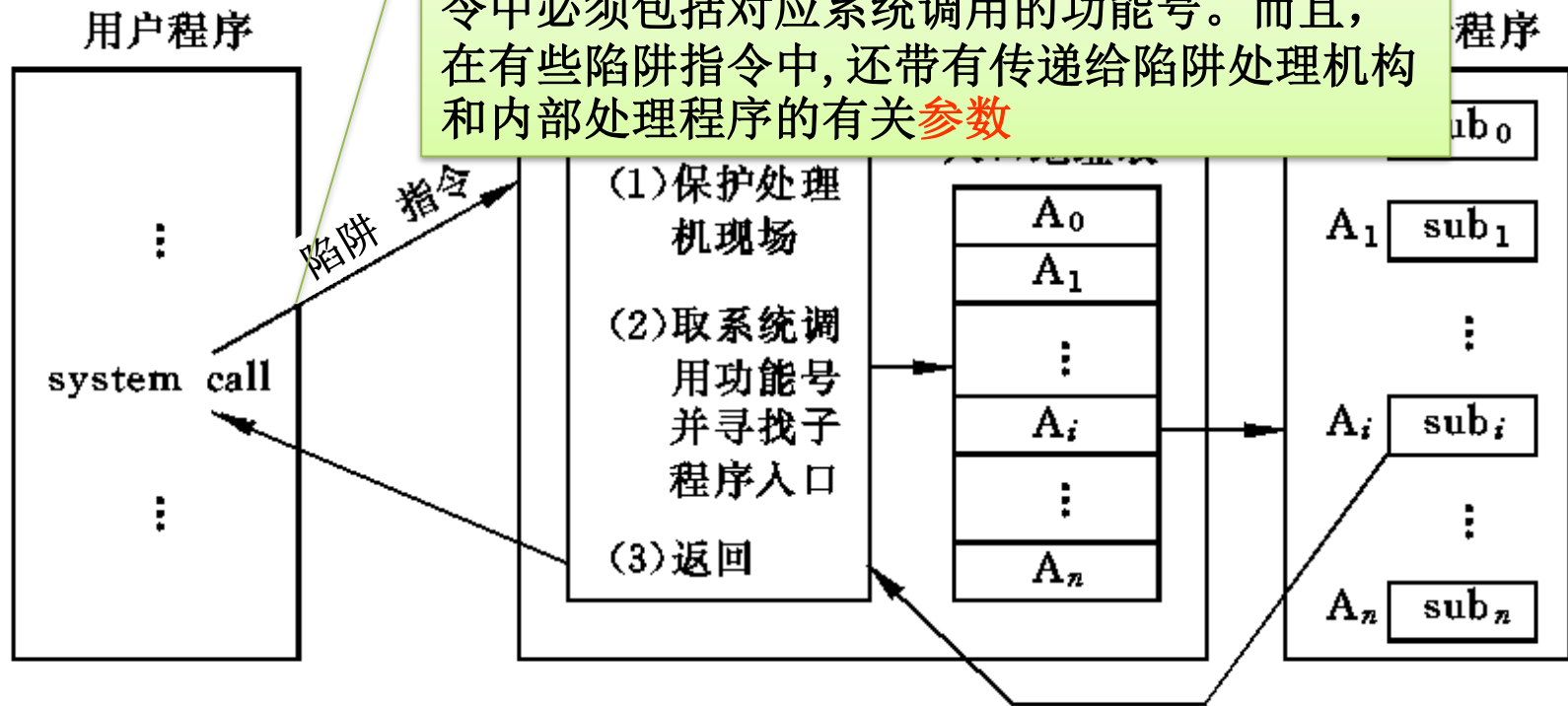
■ 最终展开成包含int指令的代码...

```
#include <unistd.h>      在linux/lib/write.c中
_syscall3(int, write, int, fd, const char
*buf, off_t, count)
```

```
#define _syscall3(type, name, ...) type
name(...) \                在linux/include/unistd.h中
{ __asm__ ("int 0x80" : "=a" (__res) ... }
```

系统调用

在操作系统中，每个系统调用都对应一个事先给定的**功能号**，例如0, 1, 2, 3等。在陷阱指令中必须包括对应系统调用的功能号。而且，在有些陷阱指令中，还带有传递给陷阱处理机构和内部处理程序的有关**参数**



用户程序和系统程序之间的参数传递

- 一条陷阱指令的长度总是有限的,而且,该指令还要携带一个系统调用的功能号,从而,陷阱指令只能自带极有限的几个参数进入系统内部

陷阱指令自
带参数

1

- 这些寄存器应是系统程序和用户程序都能访问的。不过,由于寄存器长度也是较短的。从而无法传递较多的参数

通用寄存器
传递参数

2

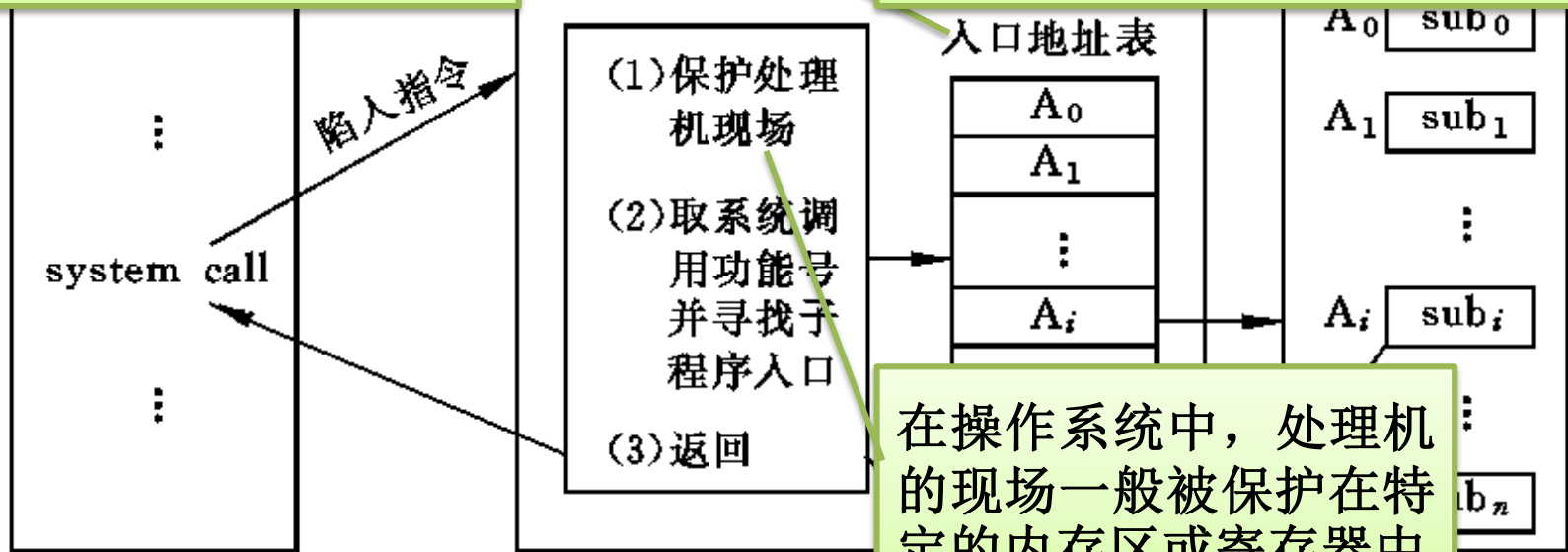
- 在系统调用较多的系统中,大多在内存中开辟专用堆栈区来传递参数

专用堆栈传
递参数

3

实现各种系统调用功能的子程序编造入口地址表，每个入口地址都与相应的系统子程序名对应起来

由陷阱处理程序把陷阱指令中所包含的功能号与该入口地址表中的有关项对应起来，从而由系统调用功能号驱动有关系统子程序执行



在操作系统中，处理机的现场一般被保护在特定的内存区或寄存器中

INT 的实现细节

➤ 请同学们课下观看哈尔滨工业大学李治军老师的
视频讲解：L5 系统调用的实现

➤ https://www.bilibili.com/video/BV19r4y1b7Aw?p=5&vd_source=3890c386258705c903d8eba25827411b

实验1（对应实验平台中的实验3）

➤在 Linux 0.11 上添加两个系统调用，并编写两

(1) iam()

第一个系统调用是 iam(), 其原型为:

```
int iam(const char * name);
```

完成的功能是将字符串参数 `name` 的内容拷贝到内核中保存下来。要求 `name` 的长度不能超过 23 个字符。返回值是拷贝的字符数。如果 `name` 的字符个数超过了 23，则返回“-1”，并置 `errno` 为 `EINVAL`。

在 `kernal/who.c` 中实现此系统调用。

(2) whoami()

第二个系统调用是 whoami(), 其原型为:

```
int whoami(char* name, unsigned int size);
```

它将内核中由 `iam()` 保存的名字拷贝到 `name` 指向的用户地址空间中，同时确保不会对 `name` 越界访存（`name` 的大小由 `size` 说明）。返回值是拷贝的字符数。如果 `size` 小于需要的空间，则返回“-1”，并置 `errno` 为 `EINVAL`。

也是在 `kernal/who.c` 中实现。

实验目的

- 建立对系统调用接口的深入认识;
- 掌握系统调用的基本过程;
- 能完成系统调用的全面控制;
- 为后续实验做准备。



实验报告要求

➤在实验报告中回答如下问题：

(1) 从 Linux 0.11 现在的机制看，它的系统调用最多能传递几个参数？你能想出办法来扩大这个限制吗？

(2) 用文字简要描述向 Linux 0.11 添加一个系统调用 `foo()` 的步骤。

评分标准

- 一个新增的系统调用被成功调用，并且能和用户空间交换参数，可得60分。两个都成功可得满分。
- 将实验结果截图到实验报告中

O S

1

简介及一般用户的输入输出界面

2

命令控制界面、图形控制界面

3

系统调用

4

Linux与Windows的命令控制界面

5

Linux与Windows的系统调用



Linux的命令控制界面



常用linux命令

- cp file1 file2 文件复制
- cat file1 在屏幕上打印文件
- pwd 显示当前工作目录
- ls 列出目录内容
- mkdir 创建目录
- rmdir 删除目录
- cd 改变目录
- vi 编辑文件 : wq filename 保存文件 : q! 不保存退出

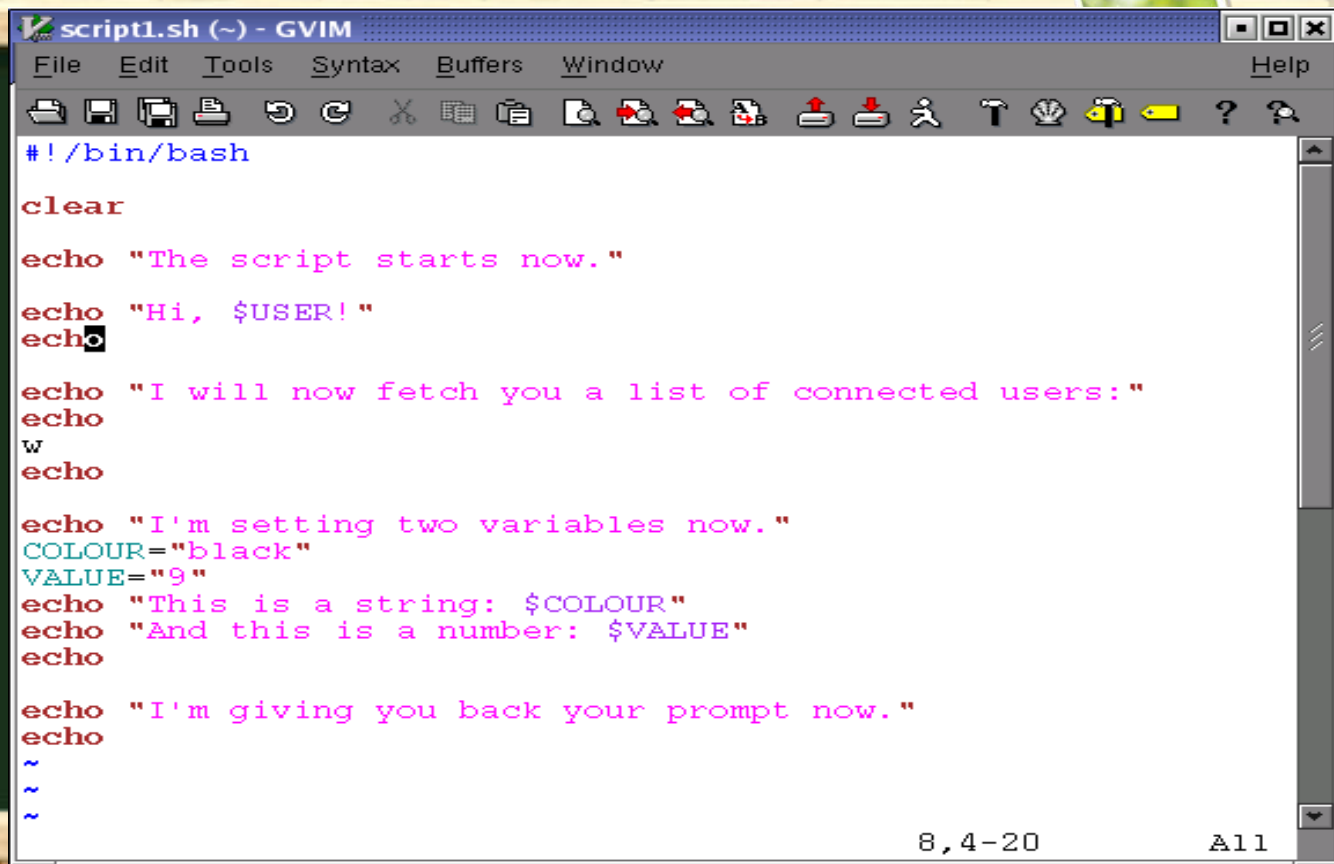


常用linux命令

- Gcc test.c 编译test.c 生成a.out 文件
- ./a.out 运行a.out
- Find / -name ls 在根目录下查找ls文件
- Grep -F test /etc/passwd 查找test用户



建立并且运行一个脚本



```
script1.sh (~) - GVIM
File Edit Tools Syntax Buffers Window Help
clear
echo "The script starts now."
echo "Hi, $USER!"
echo
echo "I will now fetch you a list of connected users:~"
echo
echo
echo "I'm setting two variables now."
COLOUR="black"
VALUE="9"
echo "This is a string: $COLOUR"
echo "And this is a number: $VALUE"
echo
echo "I'm giving you back your prompt now."
echo
~
~
~
8,4-20 All
```

执行脚本

- 正好自己写一下脚本。建立一个目录 `~/scripts` 来存放你的脚本将会是个好主意。把此目录添加到 `PATH` 变量中：
- `export PATH="$PATH:~/scripts"`

为了能使当前用户运行脚本，它应该有可执行权限。在设置权限时，检查你是否得到你想要得权限。

```
willy:~/scripts> chmod u+x script1.sh

willy:~/scripts> ls -l script1.sh
-rwxrw-r--  1 willy  willy          456 Dec 24 17:11 script1.sh

willy:~> script1.sh
The script starts now.
Hi, willy!
```

执行脚本

➤这是执行脚本最普通的方法。在子shell中一般都是这么执行脚本。建立在子shell中的变量，函数和别名等只有子shell使用。当那个shell退出，父shell重新得到控制的时候，所有的东西都被清空，脚本对shell状态所作的改变全部清除。

脚本基础

- 哪个Shell来执行脚本？
- 当在子shell运行脚本时，你应该定义哪个shell来运行脚本，你编写的脚本的shell类型可能不是你系统默认的，所以用错误的shell来运行你输入的命令可能最终出错。
- 第一行决定了启动的shell，第一行的开始2个字符应该是#!，然后紧跟解释后面命令的shell的路径。空白行也被认为是一行，所以不要让你的脚本以空白行开始。
- 例如：
#!/bin/bash
表明Bash可以在 /bin 里面找到。

加入注释

- 你应该知道事实上你不会阅读你自己脚本的唯一的一个人。很多用户和系统管理员运行别人编写的脚本。如果他们想知道你是如何做到的，注释能很好的提醒读者。
- 注释也同样让你自己更方便。你一定阅读了很多帮助页面通过脚本中的一些命令来得到特定的结果。如果不对脚本加上注释，几个星期或者几个月后你需要更改你的脚本，你会忘记脚本做了些什么事，你怎么做的和为什么要这么做。

加入

```
#!/bin/bash
# This script clears the terminal, displays a greeting and gives information
# about currently connected users. The two example variables are set and displayed.

clear                                # clear terminal window

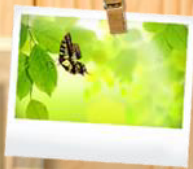
echo "The script starts now."

echo "Hi, $USER!"                    # dollar sign is used to get content of variable
echo

echo "I will now fetch you a list of connected users:"
echo
w                                     # show who is logged on and
echo                                 # what they are doing

echo "I'm setting two variables now."
COLOUR="black"                       # set a local shell variable
VALUE="9"                           # set a local shell variable
echo "This is a string: $COLOUR"     # display content of variable
echo "And this is a number: $VALUE"  # display content of variable
echo

echo "I'm giving you back your prompt now."
echo
```



Windows的命令控制界面

- Windows通过自带的**命令行解释器cmd.exe**为用户提供了功能强大的命令行控制界面，用户可以通过输入命令来对Windows操作系统进行控制和使用。
- 每个命令有不同的功能，不同的命令可以按照下面的形式进行组合从而形成新的命令。

Windows的命令控制界面

➤ **Command1 & Command2:** 用来分隔一个命令行中的多个命令。即cmd.exe运行第一个命令，然后运行第二个命令。

➤ **Command1 && Command2:** 只有在符号&&前面的命令行运行成功时，才运行符号&&后面的命令（不同的版本的windows操作系统中会有不同）。

Windows的命令控制界面

- **Command1 || Command2**: 只有在符号||前面的命令运行失败时，才运行符号||后面的命令（自己試一下）。
- **(Command1 & Command2)**: 用户分组或嵌套多个命令（自己試一下）。
- **Command1 parameter1; parameter2**: 用分号（;）分隔命令参数。

直接在命令行输入命令

- 用户运行cmd.exe进入命令行界面
- Systeminfo & mem: 显示当前系统的属性和配置等，然后显示当前内存的使用情况。



使用批处理文件

- 建立批处理文件 (.bat 或 .cmd)
- 双击批处理文件的名字

exam2. bat :

```
@ echo off  
mem > % 1 / meminfo. txt  
echo generate memoryinfo ok !
```

exam3. bat :

```
@ echo off  
type % 1 \ * . txt  
echo type ok !
```

```
@ echo off  
mdir test  
call exam2. bat test  
call exam3. bat test  
echo call ok !  
pause
```

注意： win 10 :
wmic memorychip

使用批处理文件—思考

- 将路由信息保存在test文件夹下，并在屏幕上将test文件夹下以txt为扩展名的文件内容打印出来。
- 模仿书上例题，将上述批处理文件改写为三个批处理文件，并调用（下节课提问）。

O S

1

简介及一般用户的输入输出界面

2

命令控制界面

3

系统调用

4

Linux与Windows的命令控制界面

5

Linux与Windows的系统调用



```
#include <fcntl.h>
#include <sys/stat.h>
#define SIZE 1
void filecopy( char * infile, char * Outfile)
{
    char Buffer[ SIZE ] ;
    int in_fh, Out_fh, Count;
    if( ( In_fh = open( Infile, O_RDONLY ) ) == -1 )          /* 以只读模式打开输入文件 */
        printf( "Opening infile" );
    if( ( Out_fh = open( Outfile, ( O_WRONLY | O_CREAT | O_TRUNC ),
( S_IRUSR | S_IWUSR ) ) == -1 )                             /* 以读写模式新建一个文件 */
        printf( "Opening Outfile" );
    while( ( Count = read( In_fh, Buffer, sizeof( Buffer ) ) ) > 0 ) /* 循环地向缓冲区读入输入文件内容 */
        if( write( out_fh, Buffer, Count ) != Count )
            /* 将缓冲区读入的内容写到输出文件中去 */
            printf( "Writing date" );
    if( count == -1 )
        printf( "Reading date" );
    close( In_fh );          /* 关闭输入文件 */
    close( Out_fh );        /* 关闭输出文件 */
}
```

Windows系统调用

➤ Windows操作系统也提供了丰富的系统调用。这些系统调用被进一步编写成不同的库函数后放入动态链接库（DLL）中。这些库函数构成了Windows操作系统提供给程序的编程接口（应用编程接口API）。

Windows系统调用

- 向应用程序提供了一些创建和管理用户界面的方法，可以使用窗口管理函数创建和使用窗口来显示输出、提示用户进行输入以及完成其他一些与用户进行交互所需的工作

窗口管理类

1

- 提供了一系列的函数和相关的结构，应用程序可以使用它们在显示器、打印机或其他设备上生成图形化的输出结果。

图形设备接口（GDI）类

2

- 为应用程序提供了访问计算机资源以及底层操作系统特性的手段。比如访问内存、文件系统、设备、进程和线程

系统服务类

3

Windows系统调用

- 帮助用户编写国际化的应用程序，提供给用户将应用程序本地化的一些功能。

国际特性类

4

- 允许网络上的不同计算机的应用程序之间进行通信，帮助在网络中的各计算机上创建和管理共享资源的连接。

网络服务类

5

Windows系统调用

GDI3
2.DLL

给出了屏幕显示及打印功能的函数集

USER3
2.DLL

给出了鼠标、键盘、通信端口、声音、
时钟功能的函数集

KERN
EL32.
DLL

给出了文件及内存管理（核心部分）
功能的函数集

MP.
DLL

给出了Windows 3.2网络接口库

Windows 系统调用

```
#include < windows. h >
```

```
//入口函数
```

```
int WINAPI WinMain ( HINSTANCE hInstance, HINSTANCE hPrevInstance,  
                    PSTR szCmdLine, intiCmdshow)
```

```
{
```

```
    HANDLE hFile;
```

```
    LPTSTR lpBuffer = "Hello World !";
```

```
    //创建文件
```

```
    hFile = CreateFile("C:\\File. txt", GENERIC_READ | GENERIC_WRITE,  
                      0, NULL, OPEN_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
```

```
    CloseHandle( hFile);
```

```
    TCHAR szBuf[ 128];
```

```
    DWORD dwRead;
```

```
    DWORD dwWritten;
```

```
    //打开文件
```

```
    hFile = CreateFile("C:\\File. txt", GENERIC_READ | GENERIC_WRITE,  
                      0, NULL, OPEN_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
```

Windows系统调用

//向文件中写入一个字符串

WriteFile(hFile, lpBuffer, strlen(lpBuffer) + 1, &dwRead, NULL);

SetFilePointer(hFile, 0, NULL, FILE_BEGIN);

//从文件中读出一个字符串并将它显示在对话框中

if(ReadFile(hFile, szBuf, strlen(lpBuffer) + 1, &dwWritten, NULL));

messagebox(NULL, szBuf, "EXAM", MB_OK);

CloseHandle(hFile);

}



CreateFile

The CreateFile function creates or opens the following objects and returns a handle that can be used to access the object: files pipes mailslots communications resources disk devices(Windows NT only) consoles directories(open only)

CreateFile 函数创建或打开下列对象，并返回一个可以用来访问这些对象的句柄。 文件 pipes 邮槽 通信资源 磁盘驱动器（仅适用于 windowsNT ） 控制台 文件夹（仅用于打开）

```
HANDLE CreateFile( LPCTSTR lpFileName, // 指向文件名的指针 DWORD dwDesiredAccess, // 访问模式（写 / 读） DWORD dwShareMode, // 共享模式 LPSECURITY_ATTRIBUTES lpSecurityAttributes, // 指向安全属性的指针 DWORD dwCreationDisposition, // 如何创建 DWORD dwFlagsAndAttributes, // 文件属性 HANDLE hTemplateFile // 用于复制文件句柄 );
```

Parameters 参数列表

参数	类型及说明	
lpFileName	String ，要打开的文件的名字	
dwDesiredAccess	Long ，如果为 GENERIC_READ 表示允许对设备进行读访问；如果为 GENERIC_WRITE 表示允许对设备进行写访问（可组合使用）；如果为零，表示只允许获取与一个设备有关的信息	
dwShareMode	Long ，零表示不共享； FILE_SHARE_READ 和 / 或 FILE_SHARE_WRITE 表示允许对文件进行共享访问	
lpSecurityAttributes	SECURITY_ATTRIBUTES ，指向一个 SECURITY_ATTRIBUTES 结构的指针，定义了文件的安全特性（如果操作系统支持的话）	
dwCreationDisposition	Long ，下述常数之一：	
	CREATE_NEW	创建文件；如文件存在则会出错
	CREATE_ALWAYS	创建文件，会改写前一个文件
	OPEN_EXISTING	文件必须已经存在。由设备提出要求
	OPEN_ALWAYS	如文件不存在则创建它
	TRUNCATE_EXISTING	讲现有文件缩短为零长度



WriteFile

The **WriteFile** function writes data to a file at the position specified by the file pointer. This function is designed for both synchronous and asynchronous operation.

The [WriteFileEx](#) function is designed solely for asynchronous operation.

```
BOOL WriteFile(  
    HANDLE hFile,  
    LPCVOID lpBuffer,  
    DWORD nNumberOfBytesToWrite,  
    LPDWORD lpNumberOfBytesWritten,  
    LPOVERLAPPED lpOverlapped  
);
```

Parameters

hFile

[in] Handle to the file. The file handle must have been created with the `GENERIC_WRITE` access right. For more information, see [File Security and Access Rights](#).

For asynchronous write operations, *hFile* can be any handle opened with the `FILE_FLAG_OVERLAPPED` flag by the [CreateFile](#) function, or a socket handle returned by the [socket](#) or [accept](#) function.

Windows Me/98/95: For asynchronous write operations, *hFile* can be a communications resource opened with the `FILE_FLAG_OVERLAPPED` flag by [CreateFile](#), or a socket handle returned by [socket](#) or [accept](#). You cannot perform asynchronous write operations on mailslots, named pipes, or disk files.

lpBuffer

[in] Pointer to the buffer containing the data to be written to the file.

nNumberOfBytesToWrite

[in] Number of bytes to be written to the file.

A value of zero specifies a null write operation. The behavior of a null write operation depends on the underlying file system. To truncate or extend a file, use the [SetEndOfFile](#) function.

Named pipe write operations across a network are limited to 65,535 bytes.

lpNumberOfBytesWritten

[out] Pointer to the variable that receives the number of bytes written. **WriteFile** sets this value to zero before doing any work or error checking.

If *lpOverlapped* is NULL, *lpNumberOfBytesWritten* cannot be NULL. If *lpOverlapped* is not NULL, *lpNumberOfBytesWritten* can be NULL. If this is an overlapped write operation, you can get the number of bytes written by calling [GetOverlappedResult](#). If *hFile* is associated with an I/O completion port, you can get the number of bytes written by calling [GetQueuedCompletionStatus](#).



SetFilePointer

The **SetFilePointer** function moves the file pointer of an open file.

This function stores the file pointer in two **LONG** values. To more easily work with file pointers that are larger than a single **LONG** value, use the [SetFilePointerEx](#) function.

```
DWORD SetFilePointer(  
    HANDLE hFile,  
    LONG lDistanceToMove,  
    PLONG lpDistanceToMoveHigh,  
    DWORD dwMoveMethod  
);
```

Parameters

hFile

[in] Handle to the file whose file pointer is to be moved. The file handle must have been created with the **GENERIC_READ** or **GENERIC_WRITE** access right. For more information, see [File Security and Access Rights](#).

lDistanceToMove

[in] Low-order 32 bits of a signed value that specifies the number of bytes to move the file pointer. If *lpDistanceToMoveHigh* is not NULL, *lpDistanceToMoveHigh* and *lDistanceToMove* form a single 64-bit signed value that specifies the distance to move. If *lpDistanceToMoveHigh* is NULL, *lDistanceToMove* is a 32-bit signed value. A positive value for *lDistanceToMove* moves the file pointer forward in the file, and a negative value moves the file pointer backward.

lpDistanceToMoveHigh

[in] Pointer to the high-order 32 bits of the signed 64-bit distance to move. If you do not need the high-order 32 bits, this pointer must be set to NULL. When non-NULL, this parameter also receives the high-order **DWORD** of the new value of the file pointer. For more information, see the Remarks section later in this topic.

Windows Me/98/95: If the pointer *lpDistanceToMoveHigh* is not NULL, then it must point to zero (0), **INVALID_SET_FILE_POINTER**, or the sign extension of the value of *lDistanceToMove*. Any other value is rejected.

dwMoveMethod

[in] Starting point for the file pointer move. This parameter can be one of the following values.

Value	Meaning
FILE_BEGIN	The starting point is zero (0) or the beginning of the file.
FILE_CURRENT	The starting point is the current value of the file pointer.
FILE_END	The starting point is the current end-of-file position.





MessageBox Function

The **MessageBox** function creates, displays, and operates a message box. The message box contains an application-defined message and title, plus any combination of predefined icons and push buttons.

Syntax

```
int MessageBox(  
    HWND hWnd,  
    LPCTSTR lpText,  
    LPCTSTR lpCaption,  
    UINT uType  
);
```

Parameters

hWnd

[in] Handle to the owner window of the message box to be created. If this parameter is NULL, the message box has no owner window.

lpText

[in] Pointer to a null-terminated string that contains the message to be displayed.

lpCaption

[in] Pointer to a null-terminated string that contains the dialog box title. If this parameter is NULL, the default title **Error** is used.

uType

[in] Specifies the contents and behavior of the dialog box. This parameter can be a combination of flags from the following groups of flags.

To indicate the buttons displayed in the message box, specify one of the following values.

MB_ABORTRETRYIGNORE

The message box contains three push buttons: **Abort**, **Retry**, and **Ignore**.

MB_CANCELTRYCONTINUE

Microsoft Windows 2000/XP: The message box contains three push buttons: **Cancel**, **Try Again**, **Continue**. Use this message box type instead of MB_ABORTRETRYIGNORE.

MB_HELP

Windows 95/98/Me, Windows NT 4.0 and later: Adds a **Help** button to the message box. When the user clicks the **Help** button or presses F1, the system sends a [WM_HELP](#) message to the owner.

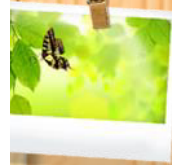
MB_OK

The message box contains one push button: **OK**. This is the default.

MB_OKCANCEL

The message box contains two push buttons: **OK** and **Cancel**.

MB_RETRYCANCEL



作业

- 复习本章所学习的内容，完成智慧树上的视频学习。
- 查阅资料，自学常用的联机控制命令
- 查阅资料，自学5类常用的API函数
- 下次课第1-2章章节小测

O S

谢谢大家！

THANK YOU!

